

# PostgreSQL Streaming Replication Recovery

Diagnosing and repairing a broken replication handshake and a read-only standby login failure caused by a login event trigger.

|          |                                                                                                           |
|----------|-----------------------------------------------------------------------------------------------------------|
| SYMPTOMS | Primary: no pg_hba.conf entry for replication · Standby: cannot execute INSERT in a read-only transaction |
| TOPOLOGY | db01 (primary) → db02 (standby) · physical streaming replication · user repuser                           |
| DATA DIR | /apps/pgsql_data/<ver> · WAL /apps/pgsql_archives · logs /apps/logs                                       |
| AUDIENCE | On-call DBA / DBRE · PostgreSQL 16 and 17+                                                                |
| VERSION  | 1.0 · June 2026                                                                                           |

## 1 • Situation & scope

Two failures are occurring at once on a primary/standby streaming-replication pair. They are **independent** but compound each other, and the second one blocks the usual fix path for the first. The roles are still correct — the primary is read-write and healthy, the standby is still in recovery — so this is a recovery, not a failover or split-brain.

### Observed symptoms

| Node           | Symptom (from logs / client)                                                                                            | What it tells us                                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary (db01) | repuser@[unknown] FATAL: no pg_hba.conf entry for replication connection from host "db02" user "repuser", no encryption | Connection reached the primary and was rejected at HBA evaluation. Network is fine; the rule no longer matches.                                             |
| Standby (db02) | FATAL: cannot execute INSERT in a read-only transaction — CONTEXT: SQL statement                                        | A write is being attempted during login on a read-only node. The CONTEXT line means the INSERT runs inside a function/trigger — i.e. a login event trigger. |

**NOTE** The two problems are causally linked for sequencing.

The permanent fix for the standby login failure lives on the **primary** (the trigger function), and it can only reach the standby *through replication* — which is currently down. That is why the restoration order in Section 6 matters: stabilise the standby locally first, restore replication second, then let the real fix replay.

### Pre-flight: confirm roles before touching anything

#### PSQL • CONFIRM TOPOLOGY

```
-- On PRIMARY (db01): should return false (read-write)
SELECT pg_is_in_recovery();

-- On STANDBY (db02): should return true (in recovery)
SELECT pg_is_in_recovery();

-- Major version drives which root cause applies (see Section 4)
SHOW server_version;
```

**CAUTION** If `pg_is_in_recovery()` is unexpected on either node, STOP.

If the primary reports `true` or the standby reports `false`, you have a different problem (failed/partial promotion). Do not proceed with this runbook — that is a failover-recovery scenario, not an HBA/trigger fix.

## 2 • Error 1 — primary rejects the replication connection

The [unknown] database is normal: replication connections do not target a real database, they target the special replication pseudo-database. The FATAL means the walreceiver reached the primary and was rejected purely at rule-matching. This is a configuration problem, not connectivity.

### Ranked likely causes

| # | Cause                                                  | Why it produces this exact message                                                                                                          |
|---|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | The replication HBA line is missing or was overwritten | A hand edit, Ansible/CM push, or template render removed the rule. Most common “what changed.”                                              |
| 2 | all used instead of the replication keyword            | A line with host all repuser ... does NOT match replication connections. The database column must be the literal keyword replication.       |
| 3 | Encryption mismatch (the no encryption tail)           | Only a hostssl line matches, but the standby connects without SSL (or vice-versa), so no rule matches and PostgreSQL appends no encryption. |
| 4 | Host / address mismatch                                | Entry uses a hostname whose reverse DNS changed, or a CIDR that no longer covers the standby's IP.                                          |
| 5 | Auth-method drift                                      | password_encryption flipped (md5 ↔ scram) or repuser's password reset, so the configured method no longer lines up.                         |

## Diagnose — read the parsed rules, not the raw file

Parsing the catalog view is faster and surfaces parse errors that the raw file hides:

ON PRIMARY (DB01)

```
-- Show every replication rule PostgreSQL actually parsed
SELECT line_number, type, database, user_name, address, auth_method, error
FROM pg_hba_file_rules
WHERE 'replication' = ANY(database);

-- Surface ANY parse error in the whole file (must return zero rows)
SELECT line_number, error FROM pg_hba_file_rules WHERE error IS NOT NULL;

-- Confirm the replication role is intact
SELECT rolname, rolcanlogin, rolreplication
FROM pg_roles WHERE rolname = 'repuser';
```

repuser must have **both** rolcanlogin = t and rolreplication = t.

## Fix — add or correct the rule, then reload

pg\_hba.conf is reload-only; no restart is needed. Choose the variant that matches your encryption posture.

PG\_HBA.CONF ON PRIMARY

```
# pg_hba.conf -- data dir: /apps/pgsql_data/<ver>/pg_hba.conf
# Option A — non-SSL replication (matches the "no encryption" client)
host replication repuser <standby_ip>/32 scram-sha-256

# Option B — SSL-enforced replication (recommended). The standby's
# primary_conninfo must then carry sslmode=require (or verify-full).
hostssl replication repuser <standby_ip>/32 scram-sha-256
```

### CAUTION Match the encryption mode on BOTH sides.

If you use hostssl here, the standby's primary\_conninfo must request SSL, otherwise you will keep getting no encryption. If the standby connects plaintext, use host. Prefer an IP/CIDR over a hostname to remove the DNS dependency entirely.

## APPLY ON PRIMARY

```
-- Reload (no restart). Run as superuser:
SELECT pg_reload_conf();

-- Re-verify: this must now return zero error rows
SELECT line_number, error FROM pg_hba_file_rules WHERE error IS NOT NULL;
```

Once the rule is correct, you do **not** need to touch the standby for Error 1. The walreceiver retries on its own (every `wal_retrieve_retry_interval`, default 5 s) and reconnects automatically.

## 3 • Error 2 — standby refuses login with a read-only INSERT

A hot-standby is read-only by design, so a plain SELECT session should connect fine. The fact that the *connection itself* dies at FATAL on an INSERT — with CONTEXT: SQL statement — means a write is being attempted **during login**, from inside a function. That signature points to a **login event trigger**.

### Mechanism

- A **login event trigger** (PostgreSQL 17+) was created on the primary — typically for login auditing — whose function performs an INSERT into an audit table.
- On the primary the INSERT succeeds. The trigger's *catalog definition* then replicates physically to the standby.
- On the read-only standby, every login attempt now fires the trigger → the INSERT fails → because it happens at login time the error is **FATAL**, locking out everyone including superusers.

**DO NOT** On the standby you cannot ALTER / DISABLE the trigger.

ALTER EVENT TRIGGER ... DISABLE is itself a catalog write, which a read-only node rejects — and you cannot connect to run it anyway. The only escape that needs no session is the file-level GUC `event_triggers`.

### This runbook covers two scenarios

| Scenario                      | Where event_triggers stands                                                                                                         | Section |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------|
| A • Trigger OFF (recovery)    | Temporarily set <code>event_triggers = off</code> on the standby so you can log in and restore service.                             | §3.1    |
| B • Trigger ON (steady state) | After the primary-side function is made standby-safe, set <code>event_triggers = on</code> again so auditing resumes on both nodes. | §3.2    |

### 3.1 • Scenario A — disable login triggers on the standby (recovery)

Goal: make the standby reachable again without waiting on replication. The `event_triggers` GUC can be set in the plain `postgresql.conf` and applied with a reload — no DB session required.

**DO NOT** Edit `postgresql.conf` — NOT `postgresql.auto.conf`.

Touching `postgresql.auto.conf` on a standby risks disturbing `primary_conninfo`. Per the replication-safety rule, never run ALTER SYSTEM RESET ALL on the standby. Set this single GUC in the plain `postgresql.conf` only.

## STANDBY · DISABLE LOGIN TRIGGERS

```
# On the STANDBY (db02). Edit the plain postgresql.conf:
sudo vi /apps/pgsql_data/<ver>/postgresql.conf

# Add (or set) this single line:
event_triggers = off

# Apply with a reload – reload does NOT need a DB session:
sudo systemctl reload postgresql
# or, if not under systemd:
pg_ctl reload -D /apps/pgsql_data/<ver>
```

Verify you can log in and the node is still a standby:

## STANDBY · CONFIRM

```
psql -U postgres -c "SELECT pg_is_in_recovery();" -- expect: t
psql -U postgres -c "SHOW event_triggers;" -- expect: off
```

**VERIFY** Success criteria for Scenario A

- psql connects on the standby without a FATAL.
- pg\_is\_in\_recovery() returns t (still a standby — you have not promoted anything).
- SHOW event\_triggers returns off.

## 3.2 · Scenario B — make the trigger standby-safe, then re-enable

Disabling event triggers on the standby is a stopgap; it also disables any other (legitimate) login triggers there. The durable fix is on the **primary**: make the audit function a no-op during recovery and never let it block a connection. Because this is a function redefinition, it replicates to the standby automatically.

## PRIMARY · STANDBY-SAFE AUDIT FUNCTION

```
-- On the PRIMARY (db01). Redefine the audit function to be standby-safe.
CREATE OR REPLACE FUNCTION login_audit()
RETURNS event_trigger
LANGUAGE plpgsql AS $$
BEGIN
-- 1) Never run the write on a standby / during recovery
IF pg_is_in_recovery() THEN
RETURN;
END IF;

INSERT INTO audit.login_log (login_time, username, application_name)
VALUES (now(), session_user, current_setting('application_name', true));

EXCEPTION
-- 2) Defence in depth: auditing must never block a login
WHEN OTHERS THEN
RETURN;
END;
$$;
```

The two guards do different jobs and you want both:

- The pg\_is\_in\_recovery() **guard** skips the write cleanly on any standby, so the trigger is harmless after promotion or on a replica.

- The **EXCEPTION WHEN OTHERS guard** ensures that even an unexpected failure (table missing, permissions, disk) degrades to “no audit row” rather than “no login.”

**NOTE** Let the redefinition replicate before re-enabling on the standby.

The new function only protects the standby once it has replayed there. Restore replication first (Section 2 / Section 6), confirm the standby is caught up, *then* flip `event_triggers` back on.

#### STANDBY · RE-ENABLE AFTER FIX REPLAYS

```
# On the STANDBY (db02), once replication is healthy and caught up:
sudo vi /apps/pgsql_data/<ver>/postgresql.conf
# set:
event_triggers = on

sudo systemctl reload postgresql

# Verify auditing is active again and logins still work:
psql -U postgres -c "SHOW event_triggers;" # expect: on
psql -U postgres -c "SELECT 1;" # connects cleanly
```

#### VERIFY Success criteria for Scenario B

- Standby logins succeed with `event_triggers = on`.
- On the primary, a login still writes an audit row; on the standby it does not (the recovery guard fires).
- `pg_event_trigger` shows the trigger enabled on both nodes.

## 4 • Version check — does this root cause even apply?

Login event triggers exist only in **PostgreSQL 17 and later**. The branch you are on changes the diagnosis for Error 2.

| Cluster version                     | Can a login fire a write?                                                          | Where to look                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 17 and later                        | Yes — a login event trigger can run an INSERT at connect time.                     | Follow Section 3 (Scenarios A and B) exactly.                                     |
| 16 and earlier<br>(your DR cluster) | No login event trigger type exists. A bare connection should not be able to write. | The write is client-side or agent-side, not server-side. See the checklist below. |

#### IDENTIFY THE TRIGGER AND VERSION

```
-- Run on BOTH nodes (primary will succeed; on standby use Scenario A first)
SELECT evtname, evt event, evt enabled FROM pg_event_trigger;
SHOW server_version;
```

If you are on PostgreSQL 16 or earlier, investigate these client-side causes of a login-time INSERT instead:

- A `~/.psqlrc` (or `system psqlrc`) that runs an INSERT / UPDATE on connect — test with `psql -X` to skip startup files.
- A connection wrapper or login script that records a session row.
- A monitoring agent, connection pooler, or backup job connecting on that socket and writing a heartbeat.

## 5 • Verify replication is actually streaming

After the HBA fix, confirm the link is live from both ends rather than assuming the automatic retry worked.

## PRIMARY • SENDER SIDE

```
-- On the PRIMARY (db01): expect state = streaming
SELECT client_addr, state, sync_state,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

## STANDBY • RECEIVER SIDE

```
-- On the STANDBY (db02): expect a live walreceiver
SELECT status, sender_host, sender_port,
       received_lsn, latest_end_lsn, last_msg_receipt_time
FROM pg_stat_wal_receiver;

-- Lag in human terms (run on standby):
SELECT now() - pg_last_xact_replay_timestamp() AS replay_delay;
```

**CAUTION** If the standby does not reconnect within ~1 minute.

Suspect a stale `primary_conninfo` (wrong host/sslmode) in `postgresql.auto.conf`. A standby restart forces a fresh attempt; on modern versions a reload restarts the walreceiver, with restart as the safe fallback. Check the standby log at `/apps/logs` for the connection error.

## STANDBY • WATCH THE RECONNECT

```
# Tail the standby log to watch the reconnect (db02):
tail -f /apps/logs/postgresql-*.log | grep -Ei 'walreceiver|streaming|primary'
```

## 6 • Order of operations (do these in sequence)

Sequence matters because the permanent trigger fix only reaches the standby through replication, which starts the procedure down.

### 1 Make the standby usable (Scenario A)

Set `event_triggers = off` in the standby's plain `postgresql.conf` and reload. Confirm `psql` connects and `pg_is_in_recovery()` is still `t`.

### 2 Fix the primary's `pg_hba.conf`

Add/correct the replication line, match the encryption mode, `SELECT pg_reload_conf()`. Confirm `pg_hba_file_rules` shows no errors.

### 3 Let the standby reconnect, then verify

The walreceiver retries automatically. Confirm `state = streaming` on the primary and a live `pg_stat_wal_receiver` on the standby (Section 5). Restart the standby only if it does not pick up within ~1 minute.

### 4 Fix the trigger on the primary (Scenario B)

Apply the standby-safe `login_audit()` redefinition on the primary. Let it replay to the standby and confirm with `pg_event_trigger`.

### 5 Re-enable event triggers on the standby

Once the new function has replayed, set `event_triggers = on` in the standby's `postgresql.conf` and reload. Confirm logins still succeed.

## 7 • Capture root-cause evidence (so it doesn't recur unexplained)

Before closing the incident, collect the “what changed” evidence:

### EVIDENCE COLLECTION

```
# When was pg_hba.conf last edited? (PRIMARY)
stat -c '%y %n' /apps/pgsql_data/<ver>/pg_hba.conf

# Any recent config-management run that could have rewritten it?
sudo grep -Ei 'pg_hba|postgres' /var/log/ansible*.log 2>/dev/null | tail
journalctl -u postgresql --since '24 hours ago' | tail -n 50

# What event triggers exist on the PRIMARY, and when created?
psql -U postgres -c "SELECT evtname, evttevent, evtowner::regrole, evtenabled
FROM pg_event_trigger ORDER BY evtname;"
```

Common true root cause: an audit/login event trigger was added on the primary without a `pg_is_in_recovery()` guard, and a separate config change (often the same maintenance window) rewrote `pg_hba.conf` and dropped the replication line. The two surface together but have independent fixes.

## 8 • Quick reference card

| Symptom                                                     | Node    | One-line fix                                                                         |
|-------------------------------------------------------------|---------|--------------------------------------------------------------------------------------|
| no <code>pg_hba.conf</code> entry for replication           | Primary | Add host replication repuser <ip>/32 scram-sha-256; SELECT pg_reload_conf()          |
| ... with no encryption                                      | Primary | Match host vs hostssl to the standby's <code>sslmode</code>                          |
| cannot execute INSERT in a read-only transaction (at login) | Standby | <code>event_triggers = off</code> in <code>postgresql.conf</code> + reload           |
| Permanent trigger fix                                       | Primary | Add IF <code>pg_is_in_recovery()</code> THEN RETURN; END IF; + EXCEPTION WHEN OTHERS |
| Re-enable auditing                                          | Standby | <code>event_triggers = on</code> after the fix replays                               |
| Confirm streaming                                           | Both    | <code>pg_stat_replication(primary) · pg_stat_wal_receiver(standby)</code>            |

### DO NOT Standby safety rules — non-negotiable

- Never ALTER SYSTEM RESET ALL on the standby (it overwrites `primary_conninfo` in `postgresql.auto.conf`). Use individual ALTER SYSTEM RESET parameter\_name only.
- When a change spans both nodes, update and restart the **standby first**, then the primary.
- For local-only GUCs like `event_triggers`, edit the plain `postgresql.conf`, not `auto.conf`.

Open items to finalise before this runbook is fully execution-ready for `ssnc-corp.cloud`: confirm the standby IP/CIDR for the HBA line, the intended encryption mode (host vs hostssl), the exact PostgreSQL major version on `db01/db02`, and the real name and target table of the login audit function if one exists.